

The *Spring Ecosystem*

Spring is not one tool — it's a family of projects. This lesson maps out the entire ecosystem so you know what exists, what each project does, and which ones you'll actually use when building microservices.

- Prerequisites: Spring Framework concepts
- Reference-style lesson — keep this as a cheat sheet
- Next lesson: Spring Boot (hands-on)

01 The Big Picture

Last lesson, we learned that Spring Framework is the **core engine** — IoC, DI, Beans, Container. But around that core, there's a whole ecosystem of projects built by the Spring team, each solving a different problem.



Think of it like a city

Spring Framework is the road system — the foundation everything is built on.

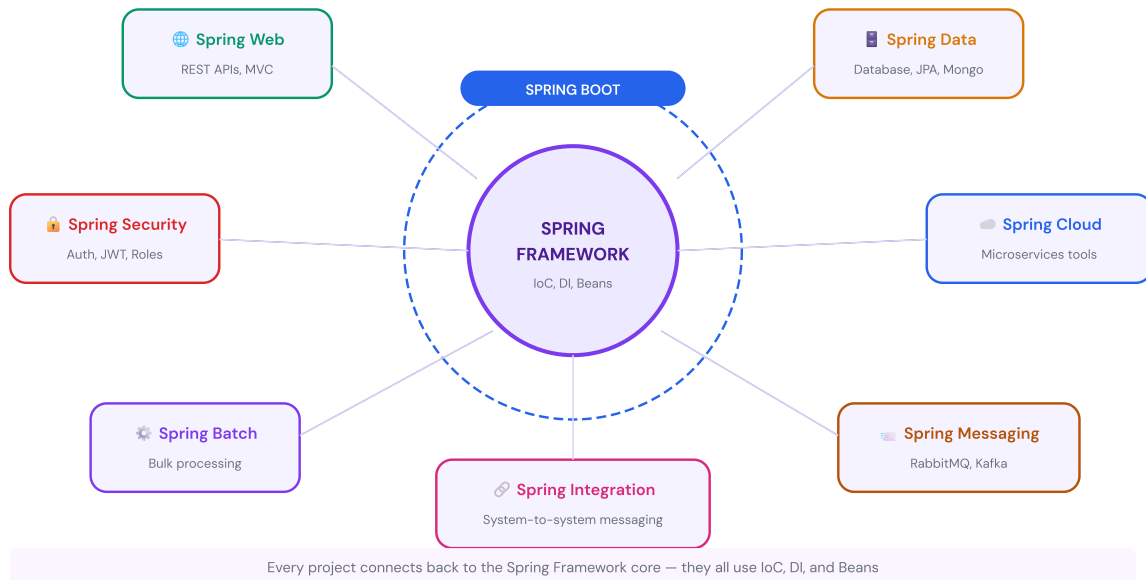
Spring Boot is the highway on-ramp — the fastest way to get moving. Around them:

Spring Data is the warehouse district, **Spring Security** is the police department,

Spring Cloud is the telecom network connecting different neighborhoods, **Spring**

Batch is the factory district running bulk operations. You don't need every district for every project — but it helps to know the map.

THE SPRING ECOSYSTEM MAP



You don't need to learn all of these. Most Java developers use only 3–5 Spring projects in their daily work. This lesson helps you understand what's available so you can pick the right tool when you need it.

02 Spring Framework & Spring Boot

These two are the foundation. You'll use them in **every** Spring project.



Spring Framework

The core engine. Provides IoC, Dependency Injection, Bean management, and the ApplicationContext container.



Spring Boot

Makes Spring easy. Auto-configuration, embedded server (Tomcat), starter dependencies, and zero XML. Turns hours of setup into minutes.

Everything else in the ecosystem builds on top of this.

Use when: Always. It's the foundation of every Spring application.

ESSENTIAL – ALWAYS USED

Use when: Always. There's almost no reason to use raw Spring Framework without Boot anymore.

ESSENTIAL – ALWAYS USED

 **Microservices Connection:** Each microservice is a Spring Boot application. When you run it, Spring Boot starts the Spring Framework container, auto-configures everything, and launches an embedded web server — all from a single `main()` method. That's the power of these two combined.

03 Spring Web (MVC & WebFlux)



Spring Web MVC

The traditional web framework. Handles HTTP requests, routing, controllers, and responses. This is what you use to build **REST APIs**. One request = one thread.

Use when: Building REST APIs or web applications. This is the default for most projects.

ESSENTIAL FOR MICROSERVICES



Spring WebFlux

The reactive alternative to MVC. Non-blocking and asynchronous — can handle thousands of connections with fewer threads. Uses a different programming model.

Use when: You need extreme scalability (streaming, real-time data, very high concurrency). Not needed for most applications.

ADVANCED – LEARN LATER

 **Microservices Connection:** Remember the REST API lesson? Spring Web MVC is the tool that turns those concepts into reality. You define controllers with URL endpoints, HTTP

methods, and JSON responses — exactly the REST patterns we learned. Every microservice uses this.

04 Spring Data

Spring Data makes database access simple. Instead of writing SQL queries and boilerplate JDBC code, you define an interface — Spring generates the implementation automatically.



Spring Data JPA

For **relational databases** (MySQL, PostgreSQL, Oracle). Uses JPA/Hibernate under the hood. Define an interface, get CRUD operations for free.

Use when: Your microservice uses a SQL database. This is the most common choice.

ESSENTIAL FOR MICROSERVICES



Spring Data MongoDB

For **MongoDB** (document database). Same pattern as JPA — define an interface, get queries automatically.

Use when: Your service stores unstructured or semi-structured data (e.g., product catalogs, user profiles).

COMMON



Spring Data Redis

For **Redis** (in-memory key-value store). Used for caching, session storage, and fast data access.

Use when: You need caching between microservices or very fast read access.

COMMON



Spring Data Elasticsearch

For **Elasticsearch** (search engine). Build powerful full-text search features.

Use when: Your service needs advanced search capabilities (e.g., product search with filters).

SPECIALIZED

 **Microservices Connection:** Remember the "database per service" rule? Each microservice chooses the database that fits its needs. Product Service might use **JPA + PostgreSQL**, while a Search Service uses **Elasticsearch**, and a Session Service uses **Redis**. Spring Data provides a consistent programming model across all of them.

05 Spring Security



Spring Security

The standard for authentication (who are you?) and authorization (what are you allowed to do?) in Spring applications. Handles login, passwords, roles, permissions, and token-based security. **Key features:** Form login, HTTP Basic, OAuth2, JWT tokens, role-based access control, CSRF protection, method-level security.

ESSENTIAL FOR PRODUCTION



Spring Authorization Server

A dedicated server that issues OAuth2 tokens. Used when you need a **centralized authentication service** that other services trust. **Use when:** You need a dedicated auth server for your microservices to validate users against.

SPECIALIZED

 **Microservices Connection:** In a microservices architecture, security is typically handled at the **API Gateway** level. The gateway validates the JWT token once, then passes the user info to downstream services. Spring Security is used in both the gateway and individual services to enforce permissions.

06 Spring Cloud

This is the **microservices-specific** part of the Spring ecosystem. Spring Cloud provides tools for the patterns we learned in the microservices lesson.



Spring Cloud Gateway

The **API Gateway**. Single entry point for all clients. Routes requests to the correct microservice, handles load balancing, rate limiting, and authentication.

ESSENTIAL FOR MICROSERVICES



Spring Cloud Netflix Eureka

Service Discovery. Services register themselves on startup. Other services find them by name instead of hardcoded URLs.

ESSENTIAL FOR MICROSERVICES



Spring Cloud Config

Centralized Configuration. All services pull their settings from one config server (backed by Git). Change config without redeploying.

COMMON



Spring Cloud Circuit Breaker

Fault tolerance. Wraps calls to other services with circuit breakers, retries, and fallbacks. Uses Resilience4j under the hood.

ESSENTIAL FOR MICROSERVICES



Spring Cloud OpenFeign

Declarative REST client. Call other microservices by writing an interface — Feign generates the HTTP client for you.

COMMON



Spring Cloud Sleuth / Micrometer Tracing

Distributed Tracing. Track a single request as it flows across multiple microservices. Find where things slow down or fail.

COMMON

 **Microservices Connection:** Remember the architecture diagram from lesson 1? API Gateway, Service Registry, Config Server, Circuit Breakers — those aren't just concepts. **Spring Cloud is the actual toolbox** that implements every one of those patterns. When we build microservices in Spring Boot, Spring Cloud is how we connect them.

07 Spring Messaging

For **asynchronous communication** between services — when you don't want to wait for a response.



Spring AMQP (RabbitMQ)

Send and receive messages through **RabbitMQ**. Ideal for task queues, event-driven communication, and decoupling services.

Use when: Services need to communicate without waiting — e.g., "order created" event triggers notification and inventory update.

COMMON IN MICROSERVICES



Spring for Apache Kafka

Send and receive messages through **Apache Kafka**. Built for high-throughput event streaming — millions of messages per second.

Use when: You need event streaming at scale — real-time analytics, log aggregation, event sourcing.

COMMON IN MICROSERVICES



Spring Cloud Stream

An abstraction layer over RabbitMQ and Kafka. Write messaging code once, switch between brokers by changing configuration.

Use when: You want flexibility to switch message brokers without changing code.

 **Microservices Connection:** In the REST lesson, we learned about synchronous (call and wait) vs asynchronous (send and forget) communication. REST handles the synchronous part. Spring Messaging handles the asynchronous part — events, notifications, background processing. Most real microservices systems use **both**.

08 Spring Batch & Spring Integration



Spring Batch

For **processing large volumes of data** in scheduled jobs. Read millions of records from a file or database, process them, and write the results — with built-in restart, skip, and retry capabilities.

Real examples: Generate monthly reports, import CSV data into a database, process nightly bank transactions, send bulk emails.

SPECIALIZED



Spring Integration

For **connecting different systems together** using Enterprise Integration Patterns. Handles message routing, transformation, and protocol adaptation (file, FTP, email, JMS, REST).

Real examples: Read files from an FTP server and process them, integrate with legacy systems, route messages between different protocols.

SPECIALIZED

 **Microservices Connection:** Not every microservice handles REST requests. Some services run **scheduled batch jobs** (e.g., a Report Service that generates daily summaries using Spring Batch). Others act as **integration bridges** between your modern microservices and older legacy systems (using Spring Integration).

09 Other Spring Projects

These are less common but worth knowing they exist. You might encounter them in specific projects.

PROJECT	WHAT IT DOES	WHEN YOU'D USE IT
Spring Session	Manages user sessions across multiple server instances	When you need session sharing between microservice instances (using Redis)
Spring HATEOAS	Adds hypermedia links to REST responses	When your API needs to guide clients to related resources via links
Spring REST Docs	Generates API documentation from tests	When you want documentation that's always in sync with your actual code
Spring State Machine	Manages complex state transitions	Order workflows (draft → submitted → approved → shipped), approval processes
Spring Shell	Build command-line applications with Spring	Admin tools, CLI utilities, interactive scripts
Spring GraphQL	Build GraphQL APIs (alternative to REST)	When clients need flexible queries — fetching exactly the data they want
Spring Modulith	Structure a monolith into well-defined modules	When you want modular monolith architecture before splitting into microservices

10 Which Ones Do I Actually Need?

Here's the practical guide. For building Java microservices, here's what you need to know and when:

Must Learn (You'll use these daily)

PROJECT	WHY
Spring Boot	Every microservice starts with this
Spring Web MVC	Every REST endpoint uses this
Spring Data JPA	Every service that touches a database uses this
Spring Security	Every production service needs authentication

Learn When Building Microservices

PROJECT	WHY
Spring Cloud Gateway	You need an API Gateway
Spring Cloud Eureka	Services need to discover each other
Spring Cloud Circuit Breaker	Handle failures between services
Spring AMQP or Kafka	Async communication between services

Learn When You Need It

PROJECT	WHY
Spring Batch	You have a bulk processing job
Spring Integration	You're connecting to legacy systems
Spring WebFlux	You need reactive, non-blocking I/O
Spring State Machine	You have complex workflow states

"I need to..." — Quick Decision Guide

"I need to build a REST API that reads from a database."

→ [Spring Boot](#) + [Spring Web MVC](#) + [Spring Data JPA](#)

"I need services to discover each other and communicate."

→ [Spring Cloud Eureka](#) + [Spring Cloud OpenFeign](#)

"I need a single entry point that routes to different services."

→ [Spring Cloud Gateway](#)

"I need to send events between services without waiting."

→ [Spring AMQP \(RabbitMQ\)](#) or [Spring Kafka](#)

"I need to handle it gracefully when a service goes down."

→ [Spring Cloud Circuit Breaker \(Resilience4j\)](#)

"I need to process a million CSV rows every night."

→ [Spring Batch](#)

"I need to secure my API with JWT tokens."

→ [Spring Security](#)

"I need to integrate with an old system that reads files from FTP."

→ [Spring Integration](#)

Official Documentation Links

Bookmark these — they're the official references from **spring.io**. When in doubt, the docs are always the most accurate and up-to-date source. Full URLs are shown so they work whether you're reading this on screen or on paper.

PROJECT	URL (TYPE INTO YOUR BROWSER)
Spring Framework	https://docs.spring.io/spring-framework/reference/
Spring Boot	https://docs.spring.io/spring-boot/docs/current/reference/html/
Spring Data JPA	https://docs.spring.io/spring-data/jpa/reference/html/
Spring Security	https://docs.spring.io/spring-security/reference/
Spring Cloud	https://spring.io/projects/spring-cloud
Spring Cloud Gateway	https://docs.spring.io/spring-cloud-gateway/reference/html/
Spring AMQP	https://docs.spring.io/spring-amqp/reference/html/
Spring for Apache Kafka	https://docs.spring.io/spring-kafka/reference/html/
Spring Batch	https://docs.spring.io/spring-batch/reference/html/
Spring Integration	https://docs.spring.io/spring-integration/reference/html/
All Spring Projects	https://spring.io/projects — the full list of everything
Spring Initializr	https://start.spring.io — generate a new project in seconds

Tip for learners: You don't need to read all the docs now. Bookmark **spring.io/projects** and **start.spring.io**. When we start building with Spring Boot next lesson, we'll use start.spring.io to generate our first project.

Learning path so far: **Microservices** (architecture) → **REST API** (communication) → **Spring Framework** (core concepts) → **Spring Ecosystem** (the map — today) → next: **Spring Boot** (we finally build something!).



What to take away from this lesson

1. Spring is a family of 20+ projects — you don't need all of them.
2. Spring Framework (IoC/DI) is the core. Spring Boot makes it easy. You always use both.
3. Spring Web MVC builds REST APIs. Spring Data handles databases. Spring Security handles auth.
4. Spring Cloud is the microservices toolkit — gateway, discovery, circuit breakers, config.
5. Spring Messaging (RabbitMQ/Kafka) handles async communication between services.
6. Spring Batch and Integration are for specialized jobs — learn them when you need them.
7. *Next lesson:* We'll use **Spring Boot** to finally build our first microservice with REST endpoints — putting everything together.